

Quantum Ledger

Budgeting & Expense Management Tool

CSCI-C 323: Mobile App Development | Fall 2025

Team 7: Prabhav Gudapati & Mihir Sisodiya

Project Description

Quantum Ledger is a native Android application that unifies expense tracking, budget monitoring, and currency conversion in one platform.

Core Functionality

- Users can track daily expenses.
- Monitor spending by category..
- Access live currency exchange rates via ExchangeRate API.

Purpose

Demonstrates mastery of Android development including RESTful API consumption with Retrofit, Activity Lifecycle management, SharedPreferences data persistence, and Material Design 3 UI/UX patterns.

Target Audience

Students but particularly International students seeking intuitive financial tracking and budgeting tools.



Group Members' Contributions

Mihir Sisodiya - Backend & Logic

- API integration: Retrofit configuration, HTTP requests, ExchangeRate API handling.
- JSON parsing and data models (Transaction, Currency, Budget).
- Budget tracking algorithm with threshold detection.
- SharedPreferences for settings, transactions, and budgets.
- Currency conversion logic and offline caching.

Prabhav Gudapati - Frontend & Lifecycle

- XML layout design for 5 Activities with Material Design 3.
- RecyclerView adapter and ViewHolder implementation.
- Widget logic: Spinner, Checkboxes, DatePicker, Radio buttons.
- Lifecycle handling: onSaveInstanceState, onPause, onResume.
- Navigation: Menu system, Intents, Dialog/Snackbar feedback.

Testing Strategies



Unit Testing

Tested transaction model, budget tracking algorithm, and currency conversion logic for correctness and edge cases.



Integration Testing

Verified Retrofit API integration with ExchangeRate API, including error handling and offline caching behavior.



UI Testing

Validated all five Activities, widget interactions (Spinner, DatePicker, Checkboxes, Radio buttons), and RecyclerView functionality.



Lifecycle Testing

Confirmed onSaveInstanceState(), onPause(), and onResume() preserve and restore app state correctly during screen rotation and backgrounding.



User Feedback Testing

Tested AlertDialog delete confirmations and Snackbar UNDO functionality for proper user experience.



Data Persistence Testing

Verified SharedPreferences correctly stores and retrieves user settings, transactions, and budget limits.



Manual Testing

Comprehensive testing of all features including add/edit/delete transactions, currency conversion, budget alerts, and menu navigation.

Implementation Timeline

Phase 1: UI Framework & Navigation (Nov 17–23)

Setup repository, create XML layouts, implement Navigation
Menu and Intents, design RecyclerView layouts.

Deliverable: Navigable prototype with static content.

1

2

3

4

Phase 3: Backend Integration (Dec 1–7)

Register API key, configure Retrofit, implement currency
conversion with caching, add error handling, implement
budget tracking.

Deliverable: Working API integration.

Phase 2: Core Logic & Storage (Nov 24–30)

Implement transaction model and SharedPreferences, create
RecyclerView adapter, implement add/edit/delete, develop
category system.

Deliverable: Functional transaction management.

Phase 4: Polish & Submission (Dec 8–14)

Implement Dialog/Snackbar, test Lifecycle, UI polish, code
cleanup, create slides, prepare APK.

Deliverable: Production-ready APK and presentation.

Technical Challenges & Solutions

Throughout the development of Quantum Ledger, we encountered several technical hurdles. Our team successfully addressed these challenges, strengthening the application's stability and user experience.



Challenge: API Rate Limiting

Implemented offline caching mechanism to store currency rates locally, reducing API calls and improving app responsiveness.

Solution: Offline caching, local storage



Challenge: Activity Lifecycle Complexity

Used `onSaveInstanceState()` to preserve form data during screen rotation and `onPause()/onResume()` for seamless state management.

Solution: Lifecycle callbacks



Challenge: Data Persistence

Leveraged `SharedPreferences` for efficient local storage of user preferences, transactions, and budget settings.

Solution: `SharedPreferences` integration.



Challenge: Real-time Updates

Integrated Retrofit with error handling and automatic retry logic to ensure reliable API communication.

Solution: Retrofit with error handling.

Implementation Details

UI Design Choices

Material Design 3 framework for modern, intuitive interface.

Five Activities with specialized layouts:

- MainActivity dashboard with summary cards and RecyclerView.
- AddTransactionActivity with Spinner (categories), DatePicker, Radio buttons (Expense/Income), Checkboxes (Tax Deductible, Recurring).
- TransactionHistoryActivity with edit/delete functionality.
- CurrencyConverterActivity for real-time conversion.
- AboutActivity for app information.

Color-coded amounts (red for expenses, green for income) for quick visual recognition.

Backend Development

- Retrofit for RESTful API consumption with ExchangeRate API integration.
- SharedPreferences for local data persistence (currency, budget limits, theme, session data).
- JSON parsing and data models (Transaction, Currency, Budget).
- Real-time currency rates with offline caching.
- Budget tracking algorithm with threshold detection.
- Activity Lifecycle management: onSaveInstanceState() for form data preservation, onPause() for transaction saving, onResume() for currency rate refresh.